CONTENT ARCHITECTURE · PROPOSAL

A composable CMS for What's New

Every feature is assembled from **blocks**. A simple update might be a hero and a gallery; a flagship gets an accordion, a stat wall, a highlight rail and a closing statement. Editors compose the page from a menu of sections — each block carrying both its content and its presentation options.

CMS	BACKEND	SOURCE OF TRUTH
Sveltia Git-based	GitHub · Vercel	content/features/*.md

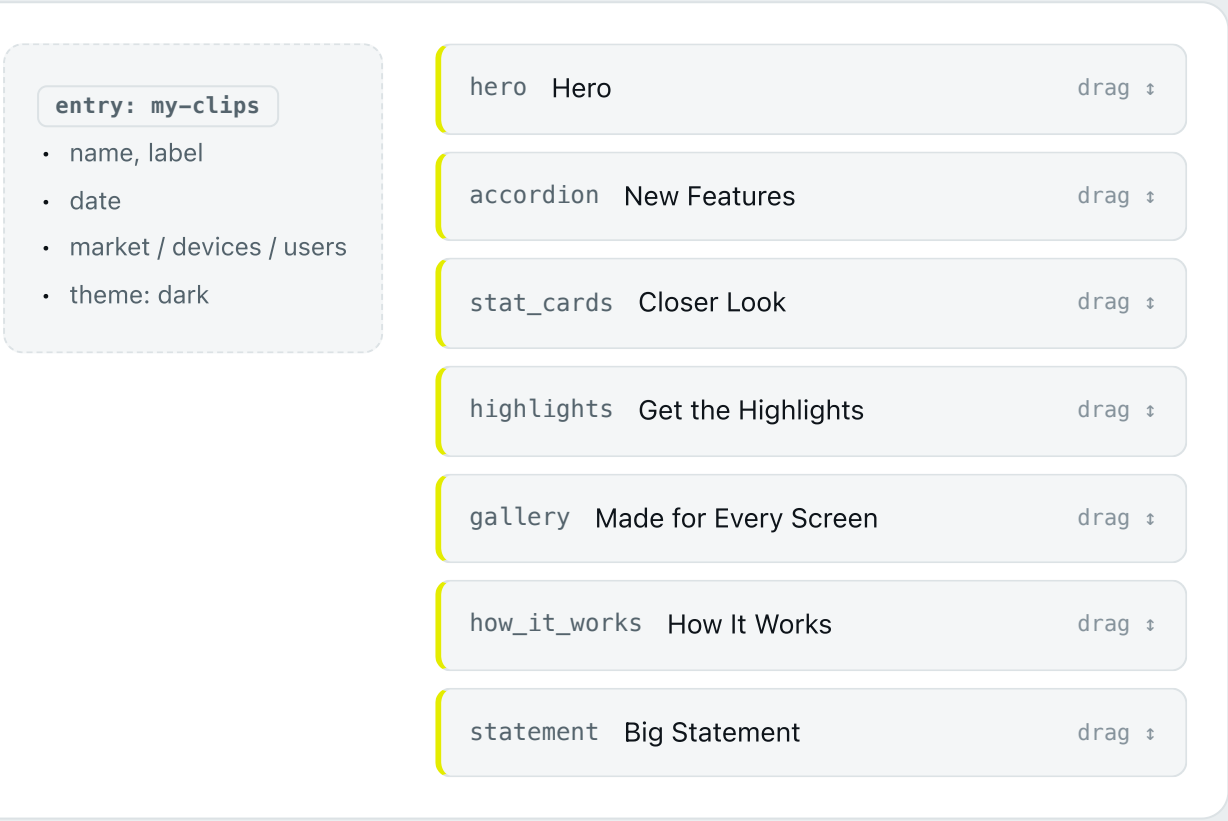
STATUS

Schema & plan

01 The model in one idea

A feature is not a fixed form. It is an identity plus an ordered list of blocks.

Today `entries.js` gives every feature the same flat set of fields, and the polish lives in hand-tuned CSS. The composable model flips that: a small set of **shared identity fields** stays constant, and the body becomes a **reorderable stack of blocks** the editor picks from. Add a highlights rail when the story needs it; leave it out when it doesn't. The template renders whatever blocks are present, in order.



Why this fits Sveltia. Decap/Sveltia's `list` widget with `types` is a native variable-block editor — editors get an "Add section" menu, drag to reorder, and each block type shows only its own fields. The content model and the editing UI are the same object.

02 Shared identity

The fields every feature carries, independent of which blocks it uses.

id	URL slug — drives `feature.html?id=…` and the filename.
name	Feature name, used in the hero eyebrow and the landing grid.
label	Update type — text (“New release”) + kind (new / improvement / live / integration) for the pill colour.
date	Release date shown in hero metadata.
market · devices · users	Metadata chip lists — the audience vocabulary (United Kingdom, Mobile, Subscriber…).
theme	light or dark — recolours the whole page through the token set.
blocks	The ordered section stack. Everything visual below lives here.

03 The block library

Each block splits into **content** (what editors write) and **options** (how it presents) — the presets currently buried in CSS become toggles.

FEATURE-PAGE BLOCKS

`hero` Hero

Opening full-bleed band: product shot, headline, benefit line, metadata chips and CTA.

CONTENT

`headline` `sub` `cta` `image` `imageMobile` `product`

OPTIONS

- `background: white / panel`
- `focal point x/y`
- `scrim: none / white / black`
- `portrait-mobile`

`accordion` New Features

Expandable list of changes with a synced image that crossfades as each item opens.

CONTENT

`items[]: heading + body` `images[]` `tip`

OPTIONS

- `crossfade tint: black / white`
- `media aspect`
- `show “Good to know”`

`stat_cards` Closer Look

Headline numbers that roll up like an odometer, each over a drawing arc gauge.

CONTENT

`cards[]: value + suffix + label` `gauge %`

OPTIONS

- `odometer shuffle`
- `gauge gradient`

`highlights` Get the Highlights

Horizontal, snap-scrolling cards — the most flexible block.

CONTENT

`cards[]: title + description` `images[]`

OPTIONS

- `layout: overlay / below / inside`
- `3-up or 4-up`
- `fill / contain + focal`
- `square < 480px`

`gallery` **Made for Every Screen**

A 2×2 device grid that animates in from the four corners on scroll.

CONTENT

`caption` `images[4]`

OPTIONS

- `per-tile fit: cover / contain`
- `per-tile background`

`gauges` **By the Numbers**

270° arc meters that draw in and count up when they hit the viewport centre.

CONTENT

`gauges[]: target + % + label`

OPTIONS

- `gradient`
- `number format`

`how_it_works` **How It Works**

Two side-by-side explainer cards: image over a short caption.

CONTENT

`cards[2]: title + body + image`

OPTIONS

- `columns`
- `media aspect`

`statement` **Big Statement**

Closing full-bleed image with one overlaid line — the emotional sign-off.

CONTENT

`eyebrow` `headline` `lead` `image`

OPTIONS

- `text position`
- `scrim strength + tint`

LANDING-PAGE BLOCKS

`feature_band` **Feature Band**

Full-width stacked band on `index.html` — the top-billed updates.

CONTENT

`name` `tagline` `cta → feature` `background` `backgroundMobile`

OPTIONS

- `treatment: white photo / dark scrim`
- `parallax`
- `mobile = whole-card link`

`tile` **Highlight Tile**

Card in the 2-up landing grid, linking through to its feature page.

CONTENT

`tileName` `tileTagline` `image` `link`

OPTIONS

- `card: light / dark`
- `hover zoom`

04

How it reads in Sveltia

The whole model is one collection with a variable-type `blocks` list. Here's the shape, with the hero block expanded.

public/admin/config.yml

```yaml
# Sveltia reads the Decap config format
backend:
  name: github
  repo: tokens25/new-features
  branch: second-main
media_folder: assets/img
public_folder: /assets/img

collections:
  - name: features
    folder: content/features
    create: true
    slug: '{{id}}'
    fields:
      - { name: id,    widget: string }
      - { name: name,  widget: string }
      - { name: theme, widget: select, options: [light, dark] }
      - { name: market, widget: list }
      # ---- the composable body ----
      - name: blocks
        label: Sections
        widget: list
        types:
          - name: hero
            widget: object
            fields:
              - { name: headline, widget: string }
              - { name: sub,      widget: text }
              - { name: image,    widget: image }
              - { name: imageMobile, widget: image, required: false }
              - { name: background,  widget: select, options: [white, panel] }
              - { name: scrim,       widget: select, options: [none, white, black]
              - { name: focalY,      widget: number, required: false }
          - { name: accordion,    widget: object, fields: […] }
          - { name: highlights,   widget: object, fields: […] }
          - { name: gallery,      widget: object, fields: […] }
          - { name: statement,    widget: object, fields: […] }
```

05 Migrating today’s content

Nothing is thrown away — the current entries convert straight into block stacks.

A one-off script reads `window.DAZN_ENTRIES` and emits one `content/features/<id>.md` per feature. Today’s flat fields map cleanly: `heroImage`→`hero`, `sections[]+closerImages[]`→`accordion`, `highlightItems[]`→`highlights`, `galleryImages[]`→`gallery`, `worksImages[]`→`how_it_works`, `statementImage`→`statement`. The render layer barely changes: a tiny build step compiles the content files back into the array the templates already consume.

> **The real work isn’t the CMS.** It’s promoting this session’s hand-tuned CSS — `object-position` crops, per-page scrims, aspect overrides — into block *options* the templates read from data. That’s where composability actually pays off, and it’s the bulk of the effort.

06 Build plan

Four phases, front-loaded on making the templates fully data-driven.

1 Content migration

Split `entries.js` into per-feature files and a build step that recompiles them. The site renders identically — pure refactor, zero visual change.

`~2–3 days` `low risk`

2 Data-drive the polish

Move focal points, scrims, aspect ratios and layout variants out of bespoke CSS and into block option fields the templates consume. The heaviest phase — and what makes the blocks truly reusable.

`~1–2 weeks` `core effort`

3 Sveltia config

Define the collection and every block type, mirroring the schema; point media at `assets/img`; add the `/admin` route.

`~2–3 days`

4 Auth, preview & deploy

GitHub sign-in (Sveltia handles the OAuth directly), live preview pointed at the real templates, and the editor live on Vercel.

`~1–2 days`

Total: roughly 3–4 weeks, and it lands incrementally — after Phase 1 the site is already file-per-feature; after Phase 3 editors can add and reorder blocks through the UI.